

HostelHub

A Node.js hostel accommodation management system built on the MVC pattern

Node.js Express EJS MongoDB

Organization	Skye8 Company LTD
Intern	Afata Favour Abeh
Track	Backend Development
Chief Supervisor	Engr. Lionel A.

Contents

1. Project Overview	3
2. Problem Statement	3
3. Background	4
4. Objectives.....	5
5. Target Users.....	5
6. Proposed Solution	5
7. Features.....	6
8. Technology Stack.....	6
9. System Architecture.....	6
10. Development Timeline.....	7
11. Expected Challenges.....	8
12. Expected Outcomes.....	8
13. References.....	9

1. Project Overview

HostelHub is a small, server-rendered system for managing hostel accommodation. The kind of thing a school hostel, a university residence, or a private lodge could put to use day to day. Hostel staff use it to add hostels and rooms, register residents, assign people to rooms, check them out again, and see at a glance which rooms still have space.

I'm building it with Node.js and Express, using the Model-View-Controller pattern the capstone requires. Pages are rendered server-side with EJS, and everything is stored in MongoDB through Mongoose. Once it's working, I intend to deploy it on Render so it has a public URL that can be clicked through rather than just read about.

This document lays out the problem I'm solving, who it's for, how I plan to build it, the technology stack, the architecture, a day-by-day timeline, and what I expect to be difficult about it. It's here for sign-off before implementation starts.

Item	Detail
Project title	HostelHub: Hostel Accommodation Management System
Intern	Afata Favour Abeh
Track	Backend Development
Chief Supervisor	Engr. Lionel A.
Architecture	Model-View-Controller (server-rendered)
Duration	Two weeks (ten working days)

2. Problem Statement

A lot of small hostels are still run out of a notebook, or at best a spreadsheet someone remembers to update. The trouble is that room information and resident information usually end up living in different places, so something as basic as “which rooms still have space” or “who's actually in room 12 right now” takes longer to answer than it should.

Manual updates go wrong in predictable ways. A room gets double-booked because nobody checked the register before assigning it. A resident checks out and the paper record isn't updated for a week. Finding a specific resident means flipping through pages instead of typing a name into a search box, and once a hostel has more than a handful of rooms, that stops being a minor annoyance and starts costing real time.

What a small hostel needs, in practice, is one shared place, a simple web app, where hostels, rooms, residents and bookings all live together. Somewhere assigning a resident, checking them out, and knowing what's free doesn't depend on someone's memory or handwriting.

In short: a small hostel needs one shared system that keeps accommodation records in one place, so assigning residents, checking them out, and tracking room availability stays fast and doesn't rely on paper.

3. Background

This is a backend capstone, and the goal isn't a flashy feature list. It's proof that I can handle the less glamorous parts of backend work: model data that has real relationships between its pieces, write controllers that enforce rules instead of just moving data around, and render pages on the server the traditional way, without cutting corners on validation or authentication. Hostel accommodation fits that well. The domain is small enough to finish in two weeks, but it still has rules worth enforcing. A room can't hold more residents than its capacity allows, and a resident shouldn't be able to occupy two rooms at once.

MVC is the backbone of the project. Mongoose schemas hold the data, controllers hold the logic (who's allowed to do what, and under what conditions), and EJS templates handle the HTML. Keeping those three things separate is mostly about not making a mess. A project this size turns into one giant file pretty fast if you're not careful, and MVC is the discipline that keeps that from happening.

The stack itself wasn't something I chose freely; Node, Express, EJS, MongoDB with Mongoose, express-session with bcrypt, and Render are all mandated for the August capstone. Even so, they hold up on their own merits for this project. Express keeps routing simple. Mongoose forces the data model to be explicit instead of implicit. And Render means deployment doesn't turn into a separate fight on top of everything else.

4. Objectives

- Model users, hostels, rooms, residents and bookings as proper Mongoose schemas, not loose objects bolted together.
- Get full create, read, update and delete flows working for hostels, rooms and residents.
- Handle room assignment and resident check-out correctly, including capacity tracking.
- Keep track of which bookings are active and which are finished, and let staff see room availability at a glance.
- Enforce room-capacity rules so a room can never take more residents than it's rated for.
- Lock the app down with session-based login and hashed passwords.
- Stick to the MVC structure throughout, not just on paper.
- Get it deployed to Render with a working public URL.

5. Target Users

The system is built for the people who manage and use hostel accommodation day to day.

User group	Needs the product serves
Hostel manager (primary)	A fast way to manage hostels, rooms and residents, assign rooms, process checkouts and monitor availability.
Hostel assistant	The same operational screens under an authorized staff account, with clear validation to catch mistakes.
Resident	A maintained record of their accommodation assignment. Self-service booking isn't part of the scope here.
Administrator (future)	Advanced reporting, staff management and multi-property support. Explicitly out of scope for the two-week capstone.

Most of this is written with one person in mind: a hostel manager on a laptop, dealing with real move-ins and move-outs, who wants an interface that doesn't get in the way. That's why the whole thing is server-rendered rather than a single-page app with a lot of client-side logic. The goal is a system that's dependable and easy to reason about, not one that shows off frontend tricks it doesn't need.

6. Proposed Solution

The app itself is a fairly standard Express setup organized around MVC. A route matches an incoming URL to a controller action; the controller does the real work (checking the session, validating whatever came in from the form, applying the accommodation rules), then calls a Mongoose model to touch the database, and finally renders an EJS view. Nothing unusual here, which is sort of the point for a backend capstone.

Sessions handle access control. A staff member logs in, the session marks them as authenticated, and any route that changes data checks for that session before doing anything. Passwords go through bcrypt before they're ever stored, so there's no version of this app where a plain-text password ends up in the database.

The rule I'm most careful about is the booking logic: a resident can't be assigned to a room that's already full, a resident can't hold two active bookings at the same time, and checking someone out has to update the room's occupancy at the same moment it closes the booking. Get that wrong and the whole “availability tracking” promise falls apart, so it's the part I plan to build and test first.

7. Features

Core features (in scope for the two-week build)

- Authorized staff sign-in and sign-out with server-side sessions.
- Password hashing with bcrypt.
- Protected routes: every data-changing action (create, update, delete, assign, check-out) requires an active session.
- Add, edit, view and remove hostels.
- Add, edit, view and remove rooms, with capacity and occupancy data.
- Add, edit, view and remove residents.
- Search residents by name or student ID.
- Search rooms by room number, and filter by hostel or availability.
- Assign a resident to a room with a check-in date.
- Check out a resident, record the date, and update room occupancy.
- Prevent room assignments that would exceed capacity.
- Prevent a resident from holding more than one active booking.
- List and filter active and completed bookings.
- Server-side validation on every form, not just client-side checks.
- Sensible 404 and error pages, rendered through the same layout as the rest of the app.
- A dashboard with counts of hostels, rooms, residents and active bookings.
- Flash messages for success and error states.
- One consistent EJS layout with shared navigation and reusable partials.

Stretch features (only if time allows)

- Resident accommodation history.
- Simple occupancy reports.
- CSV export of residents or room data.

8. Technology Stack

The stack isn't up for debate. Node, Express, EJS, MongoDB with Mongoose, express-session with bcrypt, Git/GitHub and Render are all fixed by the capstone brief. Even so, each one earns its place here: Express keeps routing and middleware simple, EJS stays close enough to plain HTML that the view layer doesn't need much explaining, and Mongoose forces me to actually think through the data model instead of treating MongoDB as a place to dump whatever shows up.

Layer	Technology	Why it was chosen
Runtime	Node.js	JavaScript on the server, with a large ecosystem and solid documentation. The basis of the capstone.
Web framework	Express	Minimal routing and middleware that maps naturally onto the MVC structure.
Views	EJS	Server-side templating that stays close to plain HTML, so the view layer stays simple.

Data and modelling	MongoDB with Mongoose	Document storage with schema modelling and validation, giving clear model definitions for MVC.
--------------------	-----------------------	--

Layer	Technology	Why it was chosen
Authentication	express-session and bcrypt	Server-side sessions for access control, and hashed passwords for safe credential storage.
Hosting	Render	Managed deployment of a Node web service, with a public URL and automatic builds from GitHub.
Version control	Git and GitHub	Meaningful commits and a development history the capstone asks for.

9. System Architecture

HostelHub follows the Model-View-Controller pattern inside a single Express application, deployed on Render. A request from the browser is matched by a route to a controller. The controller checks the session where required, validates the input, applies the accommodation rules, calls the relevant Mongoose model to read or write MongoDB, and then renders an EJS view that goes back to the browser as HTML. Figure 1 sketches out that request lifecycle.

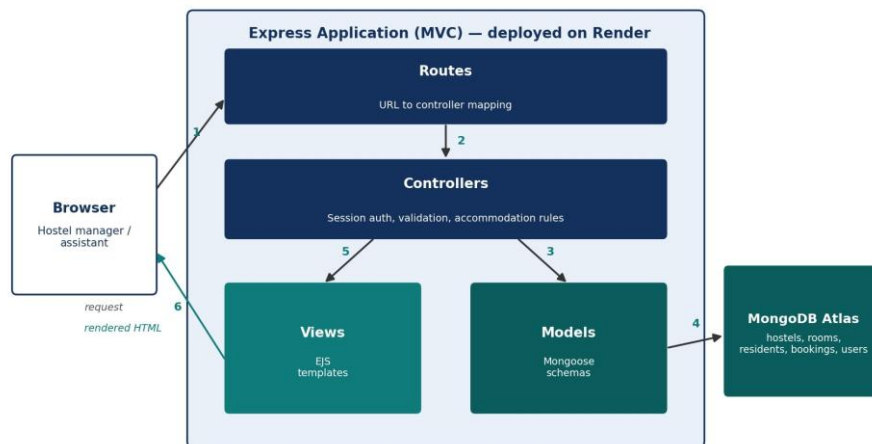


Figure 1. HostelHub's MVC request lifecycle.

Project structure (MVC)

Folder	Responsibility
/models	Mongoose schemas: User, Hostel, Room, Resident and Booking.
/controllers	Request handlers and business rules for each resource.
/routes	URL definitions mapped to controller actions.
/views	EJS templates, shared layout and reusable partials.
/middleware	Authentication guard and input validation.
app.js	Express setup, session configuration and route mounting.

Data model (high level)

Five collections carry the domain. A room belongs to a hostel, and a booking links a resident to a room, recording the dates and status that determine whether the resident is still in the room or has already checked out.

Collection	Key fields	Purpose
users	username, passwordHash, role	Authorized staff accounts for sign-in and access control.

hostels	name, location, description	An accommodation property or hostel building containing rooms.
rooms	roomNumber, hostel, capacity, occupiedSpaces	An individual room and its capacity and current occupancy.

Collection	Key fields	Purpose
residents	name, studentId, email, phone	A person who may be assigned accommodation.
bookings	resident, room, checkInDate, checkOutDate, status (active completed)	A single accommodation assignment, active until the resident checks out.

10. Development Timeline

The work is spread across the ten working days of the two-week capstone. Early days go into the MVC scaffold, the database connection and the data model; the middle stretch is authentication and the core management flows; the last few days are for testing, deployment, documentation and getting ready for the defense. Figure 2 lays that out visually, and the table underneath gives the milestone for each phase.

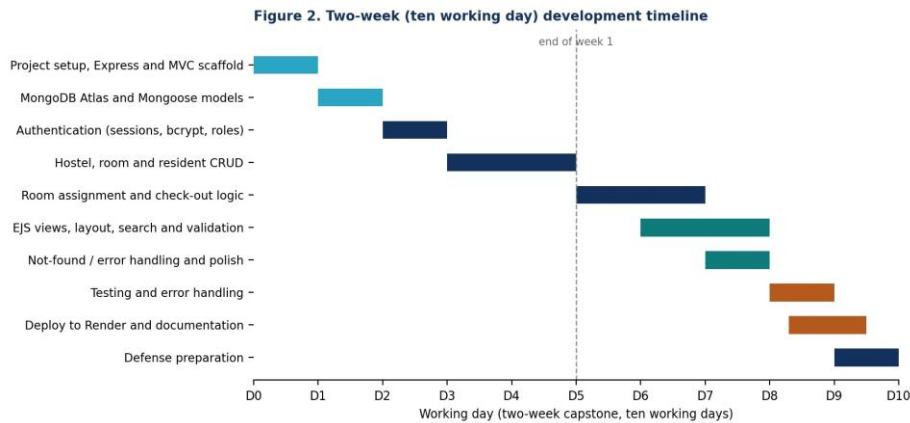


Figure 2. Two-week (ten working day) development timeline.

Days	Milestone deliverable
D0 – D1	Express project with the MVC folder structure, dependencies and a running server.
D1 – D2	MongoDB Atlas connected, and Mongoose models for User, Hostel, Room, Resident and Booking.
D2 – D3	Session-based authentication with bcrypt, and protected routes.
D3 – D5	Hostel, room and resident create/read/update/delete flows.
D5 – D7	Room assignment and check-out logic, with capacity and occupancy rules.
D6 – D8	EJS views, shared layout, validation, search/filtering, booking status views, and 404/error pages.
D8 – D10	Testing, deployment to Render, documentation and defense prep.

11. Expected Challenges

None of this is going to go perfectly on the first try, so here's what I expect to trip over, and how I plan to handle it.

Challenge	Mitigation
Keeping room occupancy correct on assignment and check-out	Only change occupiedSpaces inside the controller actions that create or close a booking, and validate capacity before assigning anyone.
Preventing duplicate active assignments	Check for an existing active booking for the resident before a new one is created.

Protecting data integrity when deleting records	Block deletion of rooms or residents that still have active bookings, or handle the related records explicitly instead of just cascading.
Secure sessions and password storage	express-session with a server-side store, bcrypt for every password. No plain-text credentials, ever.
Holding to the MVC structure under time pressure	Agree the folder layout on day one, and keep business rules in controllers and markup in views. It's tempting to shortcut this when the deadline gets close, so this is a rule I'm setting for myself early.
Fitting the scope into ten working days	Build the core features first, and treat history, reports and CSV export as optional. Only if there's time left after the core is solid.

12. Expected Outcomes

By the end of the two weeks, I expect to have:

- A deployed HostelHub application on a public Render URL.
- A GitHub repository organized clearly along MVC lines.
- Working authentication, hostel and room management, resident management, room assignment, check-out and availability tracking.
- Clear enforcement of the room-capacity and active-booking rules.
- The architecture, project structure and data model kept current with what's actually in the repository.
- Seed data (a handful of hostels, rooms, residents and bookings) so the app can be demonstrated without manual setup.
- A short README with setup and usage instructions, and a brief defense presentation.

Success, to me, comes down to one practical test: can an authorized staff member log in, manage hostels and rooms, register a resident, assign them without breaking room capacity, check them out again, and see accurate room and booking status the whole way through. On top of that, I should be able to explain, without notes, why the code is organized the way it is and what each technology choice is actually doing for the project.

13. References

The resources below are publicly available, and were chosen as the official documentation for the mandated technologies, or as recognized security guidance.

[1] OpenJS Foundation. "Node.js Documentation." nodejs.org/en/docs

- [2] OpenJS Foundation. "Express: Node.js web application framework." expressjs.com
- [3] EJS. "Embedded JavaScript templating." ejs.co
- [4] MongoDB Inc. "Mongoose Documentation." mongoosejs.com/docs
- [5] MongoDB Inc. "MongoDB Manual." mongodb.com/docs/manual
- [6] Mozilla. "HTTP." MDN Web Docs. developer.mozilla.org/en-US/docs/Web/HTTP
- [7] Render. "Render Documentation." render.com/docs
- [8] OpenJS Foundation. "express-session middleware." npm. npmjs.com/package/express-session
- [9] OWASP Foundation. "Session Management Cheat Sheet." cheatsheetseries.owasp.org
- [10] OWASP Foundation. "Password Storage Cheat Sheet." cheatsheetseries.owasp.org